Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2026-2027
Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

AUTOMATED LOG ERROR MONITORING SYSTEM

Sai Saranya – 2520030258

Varshini Bolla – 2520030431

Eesha Yuktha – 252003436

COURSE INSTRUCTOR

Dr. J Sirisha Devi
Professor
Department of Computer Science and Engineering

1. Abstract

Modern servers generate large volumes of log data, making it difficult for system administrators to manually identify and analyse errors. This project presents an **Automated Log Error Monitoring System**, implemented in Java, that detects recurring error patterns in massive server log files using efficient string-matching algorithms. Predefined error signatures are located with a Rabin–Karp rolling-hash multi-pattern scan, KMP supports ad-hoc pattern lookups, unmatched lines are grouped by edit-distance similarity, and a suffix array with an LCP array discovers new repeating patterns.

Detected errors are categorised by frequency and severity, and advanced algorithms make the results actionable: bitmask DP selects signatures under a monitoring budget, interval DP merges time buckets into report windows, job sequencing prioritises remediation, and network flow assigns tickets to on-call engineers. A 2-approximation Vertex Cover, randomised algorithms and parallel primitives complete the analysis. The system also supports continuous monitoring of log files, detecting newly generated errors in real time, and generates summary reports containing the error type, occurrence frequency and severity level.

2. Introduction

Modern servers and applications generate a large number of log files containing information about system activities, warnings and errors. Manually analysing such files is difficult, time-consuming and inefficient, and important error messages can be missed when a log contains thousands or millions of entries. Recurring errors are often the first visible sign of an underlying problem in a server, an application, a database or the network.

The proposed system automatically scans log files and detects predefined error signatures using a Rabin–Karp rolling-hash multi-pattern scan. KMP supports ad-hoc lookups, edit distance groups similar unmatched lines, and a suffix array with an LCP array discovers new patterns. Detected errors are categorised by type, frequency and severity (Low, Medium, High, Critical). Dynamic programming, network flow, Vertex Cover approximation, randomised and parallel algorithms then turn these detections into decisions, while a continuous monitor detects newly appended errors in real time and a summary report is generated.

3. Problem Statement

- Server log files can contain a very large number of entries, making manual error detection difficult.
- Important errors may be overlooked during manual log analysis.
- Recurring errors are difficult to identify without automatically tracking their frequency.
- Administrators need a simple way to categorise errors according to their severity and occurrence.

- There is a requirement for continuous monitoring to detect newly generated errors in log files.

4. Objectives

- To automatically analyse server log files.
- To detect predefined error signatures using Rabin–Karp, with KMP for single-pattern lookup.
- To cluster unmatched lines by edit distance and discover new patterns with a suffix array and LCP array.
- To identify recurring errors, calculate their frequency and classify them by severity.
- To optimise monitoring and reporting using bitmask DP, interval DP and job sequencing.
- To assign tickets with network flow and select a core monitoring set with Vertex Cover.
- To support continuous monitoring and generate a summary report of the detected errors.

5. Literature Survey

Large server logs are difficult to analyse manually, and traditional searching methods can be slow when the log files are large. The Knuth–Morris–Pratt (KMP) algorithm searches for an individual pattern in linear time by reusing information from its failure (LPS) table, while the Rabin–Karp algorithm uses a rolling hash to scan the text for one or several patterns efficiently. Edit distance measures how different two strings are and is therefore well suited to clustering similar-looking log lines, and the suffix array together with the LCP array exposes substrings that repeat in a text, which helps to discover previously unknown error patterns.

Beyond searching, dynamic programming, network flow and approximation algorithms are widely used for optimisation and assignment problems. This project combines these Data Structures and Algorithms concepts into one pipeline that automates log error detection and the decisions that follow from it.

6. Case Study: Monitoring a Production Server

Consider a web application running on a production server. Every request writes lines to a log file in the form **timestamp, level, message**, so that an hour of traffic easily produces thousands of lines. The administrator needs to know which errors occur most often, how serious they are, which new kinds of error have started to appear, and who should fix what first. Reading the file by hand is not realistic, so the system is run on the log file (for example, java logmon.Main data/sample_server.log) and works through it automatically.

6.1 Known error signatures

The system ships with eight predefined error signatures. Each signature has an identifier, the text pattern searched for, a severity and a monitoring cost (used later by the bitmask DP).

ID	Pattern searched	Severity	Cost
NULL_POINTER	NullPointerException	HIGH	3

ID	Pattern searched	Severity	Cost
CONN_REFUSED	Connection refused	HIGH	4
TIMEOUT	Timeout waiting for response	MEDIUM	5
AUTH_FAILED	Authentication failed	HIGH	3
OOM	OutOfMemoryError	CRITICAL	2
DISK_FULL	Disk full	CRITICAL	2
NOT_FOUND_404	404 Not Found	LOW	2
DEPRECATED_API	Deprecated API used	LOW	2

6.2 Worked examples

The table below shows how different kinds of line travel through the system. The first two lines follow the format of the live-monitoring demo; the last two are taken from the actual run on data/sample_server.log.

Illustrative log line	Handled by	Outcome
2026-10-05 10:15:02 ERROR NullPointerException at com.app.LiveDemo.run(LiveDemo.java:9)	Rabin–Karp multi-pattern scan	Matched as NULL_POINTER (HIGH); frequency incremented; event recorded with method RABIN-KARP.
2026-10-05 10:15:40 ERROR OutOfMemoryError: Java heap space in worker-thread-9	Rabin–Karp multi-pattern scan	Matched as OOM (CRITICAL); highest-priority remediation deadline.
WARN Cache eviction storm detected on node-7, evicted 5200 keys (4 similar lines in sample_server.log)	Edit-distance clustering (similarity ≥ 0.82)	No known signature matches; the similar lines fall into one FUZZY-CLUSTER (MEDIUM).
WARN Cache eviction storm detected on node-7, evicted 5 (repeated substring, length 55)	Suffix array + LCP array	Reported as a candidate new error signature from the unmatched lines.

6.3 From detections to decisions

Once the errors are counted, the system answers the administrator's operational questions. If only half of the total monitoring cost can be afforded (a budget of 11 out of 23 for the default signatures), bitmask DP chooses the most valuable subset of signatures to monitor. Interval DP merges the 5-minute time buckets into a small number of report windows. Job sequencing orders the remediation tasks so that CRITICAL problems (deadline slot 1) are handled before HIGH (2), MEDIUM (3) and LOW (4) ones. Finally, network flow assigns the error tickets to the available engineers:

Engineer	Daily capacity	Error categories handled
Asha (backend)	3	BACKEND (NULL_POINTER, TIMEOUT)
Ravi (infra)	2	INFRA (CONN_REFUSED, OOM, DISK_FULL)
Kiran (security)	2	SECURITY (AUTH_FAILED)
Priya (on-call / misc)	4	MISC (NOT_FOUND_404, DEPRECATED_API) and also BACKEND, INFRA, SECURITY

The maximum flow tells how many tickets can be assigned within today's capacities, and the min-cut identifies the bottleneck engineers whose capacity is fully used. In addition, an error-correlation graph is built from errors that occur in the same time bucket, and a 2-approximation Vertex Cover gives a small core set of error types that is enough to watch to cover every correlation.

7. System Modules and Course Outcome Mapping

CO	Technique	Used in the project for
CO1	Sequence alignment (edit distance); job sequencing (NP-hard scheduling class)	Clustering similar unmatched lines; remediation priority
CO2	Rabin–Karp, KMP, suffix array + LCP array	Signature detection; full-corpus lookup; discovery of new patterns
CO3	Bitmask DP; interval DP	Signature selection under a budget; merging time buckets into report windows
CO4	Network flow (Edmonds–Karp), max-flow/min-cut	Ticket assignment to engineers; bottleneck detection
CO5	NP-hard problem recognition; Vertex Cover 2-approximation	Core monitoring set from the error-correlation graph
CO6	Randomised algorithms (Las Vegas, Monte Carlo); parallel primitives	Median frequency; sampled error-rate estimate; parallel reduce and prefix sum

8. Hardware and Software Requirements

Hardware requirements

- Computer or laptop
- Minimum 4 GB RAM
- Minimum dual-core processor
- Storage space for log files and generated reports

Software requirements

- Operating system: Windows, Linux or macOS
- Programming language: Java (JDK 14 or later)
- IDE: VS Code / IntelliJ IDEA / Eclipse
- Sample server log file for testing (data/sample_server.log)
- Java standard libraries: java.nio.file, java.time and Fork/Join

ALGORITHM / PSEUDO CODE

1. Overall Workflow

The system is a pipeline. Each step feeds the next, and the final report combines all the results.

Step 1 – Load: read the log file, skip blank lines, and keep the line number, timestamp, severity and message of every entry.

Step 2 – Detect (Rabin–Karp): scan every message for all predefined signatures with a rolling hash; each hit increments the signature frequency and creates an error event.

Step 3 – Ad-hoc lookup (KMP): search the whole log corpus for a single pattern (for example OutOfMemoryError) in linear time.

Step 4 – Cluster (edit distance): group the lines no signature matched into fuzzy clusters by similarity.

Step 5 – Discover (suffix array + LCP): find substrings that repeat in the unmatched lines and propose them as new signatures.

Step 6 – Optimise (bitmask DP): choose the best subset of signatures that fits the monitoring budget.

Step 7 – Bucket and merge (interval DP): count events per 5-minute bucket and merge the buckets into report windows with minimum total cost.

Step 8 – Prioritise (job sequencing): schedule remediation tasks by profit and deadline.

Step 9 – Assign (network flow): assign tickets to engineers with Edmonds–Karp and find the min-cut bottlenecks.

Step 10 – Core set (Vertex Cover): build the error-correlation graph and compute a 2-approximate vertex cover.

Step 11 – Randomised and parallel analysis: median frequency (Las Vegas), sampled error rate (Monte Carlo), parallel reduce and prefix sum.

Step 12 – Report: render the summary (error type, frequency, severity) and write it to out/summary_report.txt.

Step 13 – Monitor: watch the log file continuously and detect newly appended errors in real time.

2. Pseudo Code

2.1 Main pipeline

```
ALGORITHM LogErrorMonitor(logFile)
  signatures <- buildKnownSignatures()
  entries    <- loadLog(logFile)          // skip blank lines, keep line no.
  KMP_Lookup(entries, "OutOfMemoryError")
  events <- [ ];  unmatched <- [ ]
  FOR each entry IN entries DO
      hits <- RabinKarp_SearchMultiple(entry.message, patternById)
      matched <- false
      FOR each (id, positions) IN hits DO
          IF positions is not empty THEN
              signatures[id].frequency++
```

```
                events.add(ErrorEvent(id, severity, time, line, msg, "RABIN-KARP"))
                matched <- true
        IF NOT matched AND message is not blank THEN unmatched.add(entry)
    events <- events + EditDistanceCluster(unmatched)
    SuffixArrayDiscovery(unmatched)
    BitmaskSelect(signatures)
    buckets  <- ComputeBuckets(entries, events)
    segments <- IntervalMerge(buckets)
    summaries <- Summarise(events)
    schedule <- JobSequencing(summaries)
    assignment <- NetworkFlowAssign(summaries)
    coreSet <- VertexCover2Approx(buckets)
    RandomizedAnalysis(summaries, entries);  ParallelAnalysis(buckets, events)
    report <- Render(...);  WRITE report TO out/summary_report.txt
    LiveMonitoringDemo(logFile)
END
```

2.2 Rabin–Karp multi-pattern search (CO2)

```
ALGORITHM RabinKarp(T, P)              // base d, prime modulus q
    n <- |T|;  m <- |P|;  IF m > n THEN RETURN [ ]
    h <- d^(m-1) mod q
    p <- hash(P);  t <- hash(T[0..m-1])
    FOR s <- 0 TO n-m DO
        IF p = t AND T[s..s+m-1] = P THEN record s    // verify on hash match
        IF s < n-m THEN
            t <- (d * (t - T[s]*h) + T[s+m]) mod q   // rolling hash update
    RETURN recorded positions

ALGORITHM searchMultiple(T, patternById)
    FOR each (id, P) IN patternById DO  result[id] <- RabinKarp(T, P)
    RETURN result
```

2.3 Knuth–Morris–Pratt search (CO2)

```
ALGORITHM ComputeLPS(P)
    lps[0] <- 0;  len <- 0;  i <- 1
    WHILE i < m DO
        IF P[i] = P[len] THEN len++;  lps[i] <- len;  i++
        ELSE IF len > 0 THEN len <- lps[len-1]
        ELSE lps[i] <- 0;  i++

ALGORITHM KMP(T, P)
    lps <- ComputeLPS(P);  i <- 0;  j <- 0
    WHILE i < n DO
        IF T[i] = P[j] THEN i++;  j++
        IF j = m THEN record (i - j);  j <- lps[j-1]
        ELSE IF i < n AND T[i] != P[j] THEN
            IF j > 0 THEN j <- lps[j-1] ELSE i++
```

2.4 Edit-distance clustering of unmatched lines (CO1)

```
ALGORITHM EditDistance(a, b)
    dp[i][0] <- i;  dp[0][j] <- j
    dp[i][j] <- MIN( dp[i-1][j] + 1,  dp[i][j-1] + 1,
                     dp[i-1][j-1] + (a[i] != b[j] ? 1 : 0) )
    RETURN dp[|a|][|b|]
    // normalised similarity = 1 - distance / max(|a|, |b|)

ALGORITHM EditDistanceCluster(unmatched)         // threshold = 0.82
    representatives <- [ ];  counts <- [ ]
    FOR each line IN unmatched DO
        best <- cluster with highest similarity(representative, line)
        IF best exists AND similarity >= 0.82 THEN
```

```
          counts[best]++;  add event "FUZZY-CLUSTER-best" (MEDIUM)
      ELSE IF line contains "ERROR" OR "WARN" THEN
          add line as a new representative;  counts <- 1
          add event "FUZZY-CLUSTER-new" (MEDIUM)
```

2.5 Suffix array and LCP discovery (CO2)

```
ALGORITHM SuffixArrayDiscovery(unmatched)
  corpus <- message(1) + SEP + message(2) + SEP + ...   // SEP = \u0001
  SA  <- starting indices of all suffixes of corpus in sorted order
  LCP[i] <- length of longest common prefix of suffix SA[i-1] and SA[i]
  candidates <- substrings with LCP >= 12 that repeat in the corpus
  RETURN top 5 candidates by repetition       // proposed new signatures
```

2.6 Bitmask DP – signature selection under a budget (CO3)

```
ALGORITHM BitmaskSelect(signatures, budget)      // budget = totalCost / 2
  k <- number of signatures;  bestValue <- 0;  bestMask <- 0
  FOR mask <- 0 TO 2^k - 1 DO
      cost  <- SUM cost(i)  FOR every bit i set in mask
      value <- SUM value(i) FOR every bit i set in mask
      IF cost <= budget AND value > bestValue THEN
          bestValue <- value;  bestMask <- mask
  RETURN signatures in bestMask
```

2.7 Time bucketing and interval DP (CO3)

```
ALGORITHM ComputeBuckets(entries, events)         // BUCKET_MINUTES = 5
  start <- timestamp of the first entry
  FOR each event e DO
      b <- floor( minutesBetween(start, e.time) / 5 )
      bucketCounts[b]++;  eventsByBucket[b].add(e)

ALGORITHM IntervalMerge(bucketCounts)
  dp[0] <- 0
  FOR j <- 1 TO numBuckets DO
      dp[j] <- MIN over i <= j of ( dp[i-1] + windowCost(i, j) )
  RETURN segments recovered from the chosen split points, minimum cost dp[n]
```

2.8 Job sequencing – remediation priority (CO1)

```
ALGORITHM RemediationScheduling(summaries)
  FOR each summary s DO
      deadline <- CRITICAL:1, HIGH:2, MEDIUM:3, LOW:4
      profit   <- s.severity.weight * s.frequency
      tasks.add(Task(s.signatureId, profit, deadline))
  SORT tasks by profit in descending order
  FOR each task IN tasks DO
      find the latest free slot <= task.deadline
      IF such a slot exists THEN place the task there, add its profit
  RETURN scheduled tasks and total profit
```

2.9 Network flow – ticket assignment with Edmonds–Karp (CO4)

```
ALGORITHM AssignTickets(demand, categoryOf, engineers)
  BUILD flow network:
      source -> ticketType        capacity = frequency (tickets of that type)
      ticketType -> engineer      if categoryOf(ticketType) is handled by engineer
      engineer -> sink            capacity = engineer daily capacity
  WHILE BFS finds a shortest augmenting path source -> sink in residual graph
      push the bottleneck capacity along the path (update residual edges)
  totalAssigned <- max-flow value
  assignedPerEngineer <- flow on each engineer -> sink edge
```

```
  bottleneck <- engineers whose engineer -> sink edge is saturated (min cut)
  RETURN totalAssigned, assignedPerEngineer, bottleneck
```

2.10 Vertex Cover 2-approximation (CO5)

```
ALGORITHM BuildCorrelationGraph(buckets)
  FOR each bucket DO
      types <- distinct error-type ids in the bucket
      FOR every pair (u, v) in types DO  graph.addCooccurrence(u, v)

ALGORITHM VertexCoverApprox(graph)
  cover <- { };  E <- all edges of graph
  WHILE E is not empty DO
      pick any edge (u, v) IN E
      cover <- cover U {u, v}
      remove from E every edge incident to u or v
  RETURN cover                                  // |cover| <= 2 * optimum
```

2.11 Randomised algorithms (CO6)

```
ALGORITHM RandomizedSelect(A, lo, hi, k)          // Las Vegas: always exact
  IF lo = hi THEN RETURN A[lo]
  pivot <- random index in [lo, hi];  q <- Partition(A, lo, hi, pivot)
  IF k = q THEN RETURN A[q]
  ELSE IF k < q THEN RETURN RandomizedSelect(A, lo, q-1, k)
  ELSE RETURN RandomizedSelect(A, q+1, hi, k)
  // median error-type frequency, expected time O(n)

ALGORITHM MonteCarloErrorRate(lines, fraction = 0.4)
  sample <- random fraction*|lines| lines
  RETURN (lines in sample containing "ERROR") / |sample|   // approximate
```

2.12 Parallel primitives – Fork/Join (CO6)

```
ALGORITHM ParallelReduceSum(A, lo, hi)
  IF hi - lo <= THRESHOLD THEN RETURN sequential sum of A[lo..hi-1]
  mid <- (lo + hi) / 2
  left <- FORK ParallelReduceSum(A, lo, mid)
  right <- ParallelReduceSum(A, mid, hi)
  RETURN JOIN(left) + right

ALGORITHM ParallelPrefixSum(A)        // cumulative bucket counts
  compute all prefix sums with parallel passes over the array
  // work O(n log n), span O(log^2 n)
```

2.13 Continuous monitoring (real-time detection)

```
ALGORITHM LiveMonitoringDemo(originalLog)
  copy originalLog to out/live_demo.log
  start daemon watcher thread running LogMonitor.watch(callback)
      callback(line): hits <- RabinKarp_SearchMultiple(line, patternById)
                      FOR each matching signature DO print "[LIVE DETECTED] ..."
  main thread: sleep, append an ERROR line (NullPointerException), sleep,
               append an ERROR line (OutOfMemoryError), sleep, finish
```

3. Complexity Summary

Technique	Time complexity	Notes
Rabin–Karp (one pattern)	O(n + m) average, O(nm) worst	n = text length, m = pattern length

Technique	Time complexity	Notes
KMP	O(n + m)	failure (LPS) table avoids re-comparison
Edit distance	O(m × n)	per pair of lines
Bitmask DP	O(2^k × k)	k = number of signatures (8 here)
Edmonds–Karp	O(V × E^2)	small network: ticket types + engineers
Vertex Cover 2-approximation	O(V + E)	cover size at most twice the optimum
Randomised select (Las Vegas)	O(n) expected	result is always exact
Parallel reduce / prefix sum	work O(n) / O(n log n)	span O(log n) / O(log² n)

CODE

The program is written in Java (package logmon) and is organised into sub-packages for each technique. The driver class Main.java reads the log file, runs every stage of the pipeline in order and prints the results; the screenshots on the following pages show it in full.

Package	Classes	Purpose
logmon	Main	Driver: loads the log, runs the pipeline, writes the report
logmon.model	LogEntry, ErrorEvent, ErrorSignature, Severity	Data model for lines, detected events and signatures
logmon.matching	RabinKarpMatcher, KMPMatcher, EditDistanceMatcher, SuffixArrayDiscovery	String-matching and pattern-discovery algorithms
logmon.optimize	BitmaskSignatureSelector, IntervalReportMerger, RemediationScheduler	Dynamic programming and job sequencing
logmon.flow	EngineerAssignment	Network flow (Edmonds–Karp) ticket assignment
logmon.npc	ErrorCorrelationGraph, VertexCoverApproximation	NP-hard problem recognition and approximation
logmon.randomized / logmon.parallel	RandomizedAlgorithms, ParallelPrimitives	Las Vegas / Monte Carlo and Fork/Join primitives
logmon.monitor / logmon.report	LogMonitor, ReportGenerator	Real-time monitoring and report generation

Main.java 9+

logmon > src > main > java > logmon > Main.java > Main > BUCKET_MINUTES

```java
package logmon;

import logmon.flow.EngineerAssignment;
import logmon.matching.EditDistanceMatcher;
import logmon.matching.KMPMatcher;
import logmon.matching.RabinKarpMatcher;
import logmon.matching.SuffixArrayDiscovery;
import logmon.model.ErrorEvent;
import logmon.model.ErrorSignature;
import logmon.model.LogEntry;
import logmon.model.Severity;
import logmon.monitor.LogMonitor;
import logmon.npc.ErrorCorrelationGraph;
import logmon.npc.VertexCoverApproximation;
import logmon.optimize.BitmaskSignatureSelector;
import logmon.optimize.IntervalReportMerger;
import logmon.optimize.RemediationScheduler;
import logmon.parallel.ParallelPrimitives;
import logmon.randomized.RandomizedAlgorithms;
import logmon.report.ReportGenerator;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;
import java.util.*;

public class Main {

    private static final double FUZZY_SIMILARITY_THRESHOLD = 0.82;
    private static final int BUCKET_MINUTES = 5;

    Run | Debug
    public static void main(String[] args) throws Exception {
        Path logFile = Paths.get(args.length > 0 ? args[0] : "data/sample_server.log");

        List<ErrorSignature> signatures = buildKnownSignatures();
        List<LogEntry> entries = loadLog(logFile);
        System.out.println("Loaded " + entries.size() + " log lines from " + logFile);

        // ---- C02: ad-hoc single-pattern KMP lookup across the whole corpus ----
        demoKmpFullCorpusLookup(entries, pattern: pattern: "OutOfMemoryError");

        // ---- C02: Rabin-Karp multi-pattern rolling-hash scan (primary detector) ----
        Map<String, String> patternById = new LinkedHashMap<>();
        for (ErrorSignature sig : signatures) {
            patternById.put(sig.getId(), sig.getPattern());
        }
        Map<String, ErrorSignature> sigById = new HashMap<>();
        for (ErrorSignature sig : signatures) {
            sigById.put(sig.getId(), sig);
        }

        List<ErrorEvent> events = new ArrayList<>();
        List<LogEntry> unmatched = new ArrayList<>();

```

Fig. 1: Main.java – package, imports, constants and the start of main() (CO2: KMP lookup and Rabin–Karp scan setup), lines 1–58



```
60          for (LogEntry entry : entries) {
61              Map<String, List<Integer>> hits = RabinKarpMatcher.searchMultiple(entry.getMessage(), patternById);
62              boolean matchedSomething = false;
63              for (Map.Entry<String, List<Integer>> hit : hits.entrySet()) {
64                  if (!hit.getValue().isEmpty()) {
65                      ErrorSignature sig = sigById.get(hit.getKey());
66                      sig.incrementFrequency();
67                      events.add(new ErrorEvent(sig.getId(), sig.getSeverity(), entry.getTimestamp(),
68                              entry.getLineNumber(), entry.getMessage(), detectionMethod: detectionMethod: "RABIN-KARP"));
69                      matchedSomething = true;
70                  }
71              }
72              if (!matchedSomething && !entry.getMessage().isBlank()) {
73                  unmatched.add(entry);
74              }
75          }
76
77          // ---- CO1: sequence alignment (edit distance) fuzzy clustering of unmatched lines ----
78          List<ErrorEvent> fuzzyEvents = clusterUnmatchedByEditDistance(unmatched);
79          events.addAll(fuzzyEvents);
80
81          // ---- CO2: suffix array + LCP array discovery of brand-new repeating patterns ----
82          runSuffixDiscovery(unmatched);
83
84          // ---- CO3: bitmask DP signature-subset selection under a monitoring budget ----
85          runBitmaskSelection(signatures);
86
87          // ---- Bucket events by time window once, reused by CO3's interval DP and CO5's correlation graph ----
88          BucketingResult buckets = computeBuckets(entries, events);
89
90          // ---- CO3: interval DP merging of time-bucketed error counts into report windows ----
91          List<IntervalReportMerger.Segment> segments = runIntervalMerge(buckets);
92
93          // ---- CO1: NP-hard-scheduling-class problem (job sequencing) for remediation priority ----
94          List<ReportGenerator.Summary> summaries = ReportGenerator.summarise(events);
95          RemediationScheduler.ScheduleResult schedule = runRemediationScheduling(summaries);
96
97          // ---- CO4: network flow (Edmonds-Karp) + max-flow/min-cut duality for ticket assignment ----
98          EngineerAssignment.Result assignment = runEngineerAssignment(summaries);
99
100         // ---- CO5: NP-hard problem recognition (Vertex Cover) + provable-ratio approximation ----
101         Set<String> coreMonitoringSet = runVertexCoverAnalysis(buckets);
102
103         // ---- CO6: randomised algorithms - Las Vegas select + Monte Carlo sampling ----
104         RandomizedSummary randomizedSummary = runRandomizedAnalysis(summaries, entries);
105
106         // ---- CO6: parallel-algorithm primitives - parallel reduce + parallel prefix sum ----
107         ParallelSummary parallelSummary = runParallelAnalysis(buckets, events);
108
109         // ---- Final report ----
110         String coreReport = ReportGenerator.render(entries.size(), events, summaries, segments, schedule);
111         String extendedReport = renderExtendedReport(assignment, coreMonitoringSet, randomizedSummary, parallelSummary);
112         String fullReport = coreReport + extendedReport;
113
```

Fig. 2: Main.java – Rabin–Karp detection loop and the full analysis pipeline calling CO1–CO6 modules, lines 60–113

```
114         System.out.println(fullReport);
115         Path outFile = Paths.get(first: first: "out/summary_report.txt");
116         Files.createDirectories(outFile.getParent());
117         ReportGenerator.writeToFile(fullReport, outFile);
118         System.out.println("Full report written to " + outFile.toAbsolutePath());
119
120         // ---- Continuous / real-time monitoring demo ----
121         runLiveMonitoringDemo(logFile, patternById, sigById);
122     }
123
124     // ------------------------------------------------------------------
125     // Setup
126     // ------------------------------------------------------------------
127
128     private static List<ErrorSignature> buildKnownSignatures() {
129         List<ErrorSignature> list = new ArrayList<>();
130         list.add(new ErrorSignature(id: id: "NULL_POINTER", pattern: pattern: "NullPointerException", Severity.HIGH, cost: cost: 3));
131         list.add(new ErrorSignature(id: id: "CONN_REFUSED", pattern: pattern: "Connection refused", Severity.HIGH, cost: cost: 4));
132         list.add(new ErrorSignature(id: id: "TIMEOUT", pattern: pattern: "Timeout waiting for response", Severity.MEDIUM, cost: cost: 5));
133         list.add(new ErrorSignature(id: id: "AUTH_FAILED", pattern: pattern: "Authentication failed", Severity.HIGH, cost: cost: 3));
134         list.add(new ErrorSignature(id: id: "OOM", pattern: pattern: "OutOfMemoryError", Severity.CRITICAL, cost: cost: 2));
135         list.add(new ErrorSignature(id: id: "DISK_FULL", pattern: pattern: "Disk full", Severity.CRITICAL, cost: cost: 2));
136         list.add(new ErrorSignature(id: id: "NOT_FOUND_404", pattern: pattern: "404 Not Found", Severity.LOW, cost: cost: 2));
137         list.add(new ErrorSignature(id: id: "DEPRECATED_API", pattern: pattern: "Deprecated API used", Severity.LOW, cost: cost: 2));
138         return list;
139     }
140
141     private static List<LogEntry> loadLog(Path logFile) throws IOException {
142         List<String> rawLines = Files.readAllLines(logFile, StandardCharsets.UTF_8);
143         List<LogEntry> entries = new ArrayList<>();
144         int lineNo = 1;
145         for (String line : rawLines) {
146             if (!line.isBlank()) {
147                 entries.add(new LogEntry(lineNo, line));
148             }
149             lineNo++;
150         }
151         return entries;
152     }
153
154     // ------------------------------------------------------------------
155     // CO2 - KMP ad-hoc lookup
156     // ------------------------------------------------------------------
157
158     private static void demoKmpFullCorpusLookup(List<LogEntry> entries, String pattern) {
159         StringBuilder corpus = new StringBuilder();
160         for (LogEntry e : entries) {
161             corpus.append(e.getMessage()).append(c: c: '\n');
162         }
163         List<Integer> positions = KMPMatcher.search(corpus.toString(), pattern);
164         System.out.println("\n[CO2 | KMP] Full-corpus lookup for \"" + pattern + "\": "
165                 + positions.size() + " occurrence(s) at character offsets " + positions);
166     }
167
```

Fig. 3: Main.java – report writing, buildKnownSignatures(), loadLog() and the KMP full-corpus lookup, lines 114–167



```
168     // ------------------------------------------------------------------
169     // CO1 - sequence alignment clustering
170     // ------------------------------------------------------------------
171
172     private static List<ErrorEvent> clusterUnmatchedByEditDistance(List<LogEntry> unmatched) {
173         List<String> representatives = new ArrayList<>();
174         List<Integer> clusterCounts = new ArrayList<>();
175         List<ErrorEvent> fuzzyEvents = new ArrayList<>();
176
177         for (LogEntry entry : unmatched) {
178             String msg = entry.getMessage();
179             int bestCluster = -1;
180             double bestSim = 0.0;
181             for (int i = 0; i < representatives.size(); i++) {
182                 double sim = EditDistanceMatcher.similarity(representatives.get(i), msg);
183                 if (sim > bestSim) {
184                     bestSim = sim;
185                     bestCluster = i;
186                 }
187             }
188             if (bestCluster != -1 && bestSim >= FUZZY_SIMILARITY_THRESHOLD) {
189                 clusterCounts.set(bestCluster, clusterCounts.get(bestCluster) + 1);
190                 fuzzyEvents.add(new ErrorEvent("FUZZY-CLUSTER-" + bestCluster, Severity.MEDIUM,
191                         entry.getTimestamp(), entry.getLineNumber(), msg, detectionMethod: detectionMethod: "EDIT-DISTANCE"));
192             } else if (msg.toUpperCase(Locale.ROOT).contains(s: s: "ERROR") || msg.toUpperCase(Locale.ROOT).contains(s: s: "WARN")) {
193                 representatives.add(msg);
194                 clusterCounts.add(e: e: 1);
195                 fuzzyEvents.add(new ErrorEvent("FUZZY-CLUSTER-" + (representatives.size() - 1), Severity.MEDIUM,
196                         entry.getTimestamp(), entry.getLineNumber(), msg, detectionMethod: detectionMethod: "EDIT-DISTANCE"));
197             }
198         }
199
200         System.out.println("\n[CO1 | Edit-Distance] Formed " + representatives.size()
201                 + " fuzzy cluster(s) from " + unmatched.size() + " unmatched line(s) "
202                 + "(similarity threshold " + FUZZY_SIMILARITY_THRESHOLD + "):");
203         for (int i = 0; i < representatives.size(); i++) {
204             System.out.println("   FUZZY-CLUSTER-" + i + " x" + clusterCounts.get(i)
205                     + "  e.g. \"" + representatives.get(i) + "\"");
206         }
207         return fuzzyEvents;
208     }
209
210     // ------------------------------------------------------------------
211     // CO2 - suffix array / LCP discovery
212     // ------------------------------------------------------------------
213
214     private static void runSuffixDiscovery(List<LogEntry> unmatched) {
215         if (unmatched.isEmpty()) {
216             return;
217         }
218         StringBuilder corpus = new StringBuilder();
219         for (LogEntry e : unmatched) {
220             corpus.append(e.getMessage()).append(c: c: '\u0001'); // separator unlikely to appear in log text
221         }
222         SuffixArrayDiscovery discovery = new SuffixArrayDiscovery(corpus.toString());
223         List<SuffixArrayDiscovery.RepeatCandidate> repeats = discovery.topRepeats(topN: topN: 5, minLength: minLength: 12);
224
225         System.out.println("\n[CO2 | Suffix Array + LCP] Candidate NEW error signatures discovered "
226                 + "from previously unmatched lines:");
```

Fig. 4: Main.java – edit-distance clustering (CO1) and the start of suffix array + LCP discovery (CO2), lines 168–226

```
225         System.out.println("\n[CO2 | Suffix Array + LCP] Candidate NEW error signatures discovered "
226                 + "from previously unmatched lines:");
227         if (repeats.isEmpty()) {
228             System.out.println(x: x: "   (no substring repeated often enough to suggest a new signature)");
229         } else {
230             for (SuffixArrayDiscovery.RepeatCandidate c : repeats) {
231                 System.out.println("   " + c);
232             }
233         }
234     }
235
236     // ---------------------------------------------------------------
237     // CO3 - bitmask DP subset selection
238     // ---------------------------------------------------------------
239
240     private static void runBitmaskSelection(List<ErrorSignature> signatures) {
241         int totalCost = signatures.stream().mapToInt(ErrorSignature::getCost).sum();
242         int budget = totalCost / 2; // simulate a constrained-resource monitoring mode
243
244         BitmaskSignatureSelector.Selection selection = BitmaskSignatureSelector.selectBest(signatures, budget);
245         System.out.println("\n[CO3 | Bitmask DP] Resource-constrained monitoring mode (budget=" + budget
246                 + " of max " + totalCost + "):");
247         System.out.println("   Selected signatures (total cost=" + selection.totalCost
248                 + ", total value=" + selection.totalValue + "):");
249         for (ErrorSignature s : selection.chosen) {
250             System.out.println("     " + s + " severity=" + s.getSeverity()
251                     + " freq=" + s.getHistoricalFrequency() + " cost=" + s.getCost());
252         }
253     }
254
255     // ---------------------------------------------------------------
256     // Bucketing (shared by CO3 interval-merge and CO5 correlation graph)
257     // ---------------------------------------------------------------
258
259     private static class BucketingResult {
260         final int[] bucketCounts;
261         final Map<Integer, List<ErrorEvent>> eventsByBucket;
262
263         BucketingResult(int[] bucketCounts, Map<Integer, List<ErrorEvent>> eventsByBucket) {
264             this.bucketCounts = bucketCounts;
265             this.eventsByBucket = eventsByBucket;
266         }
267     }
268
269     private static BucketingResult computeBuckets(List<LogEntry> entries, List<ErrorEvent> events) {
270         LocalDateTime start = entries.get(index: index: 0).getTimestamp();
271         long maxMinutes = 0;
272         for (ErrorEvent e : events) {
273             long minutes = ChronoUnit.MINUTES.between(start, e.getTimestamp());
274             maxMinutes = Math.max(maxMinutes, minutes);
275         }
276         int bucketCount = (int) (maxMinutes / BUCKET_MINUTES) + 1;
277         int[] bucketCounts = new int[bucketCount];
278         Map<Integer, List<ErrorEvent>> eventsByBucket = new HashMap<>();
279         for (ErrorEvent e : events) {
280             long minutes = ChronoUnit.MINUTES.between(start, e.getTimestamp());
281             int bucket = (int) (minutes / BUCKET_MINUTES);
282             bucketCounts[bucket]++;
```

Fig. 5: Main.java – suffix array output, bitmask DP selection (CO3) and time bucketing, lines 225–282

```
283             eventsByBucket.computeIfAbsent(bucket, k -> new ArrayList<>()).add(e);
284         }
285         return new BucketingResult(bucketCounts, eventsByBucket);
286     }
287
288     // ------------------------------------------------------------------
289     // CO3 - interval DP report-window merging
290     // ------------------------------------------------------------------
291
292     private static List<IntervalReportMerger.Segment> runIntervalMerge(BucketingResult buckets) {
293         IntervalReportMerger merger = new IntervalReportMerger(buckets.bucketCounts);
294         List<IntervalReportMerger.Segment> segments = merger.optimalSegments();
295         System.out.println("\n[CO3 | Interval DP] " + buckets.bucketCounts.length + " time-buckets (each "
296                 + BUCKET_MINUTES + " min) merged into " + segments.size() + " report window(s), min total cost="
297                 + merger.minimumCost());
298         return segments;
299     }
300
301     // ------------------------------------------------------------------
302     // CO1 - NP-hard scheduling class (job sequencing) for remediation
303     // ------------------------------------------------------------------
304
305     private static RemediationScheduler.ScheduleResult runRemediationScheduling(
306             List<ReportGenerator.Summary> summaries) {
307         List<RemediationScheduler.Task> tasks = new ArrayList<>();
308         for (ReportGenerator.Summary s : summaries) {
309             int deadline = switch (s.severity) {
310                 case CRITICAL -> 1;
311                 case HIGH -> 2;
312                 case MEDIUM -> 3;
313                 case LOW -> 4;
314             };
315             int profit = s.severity.weight * s.frequency;
316             tasks.add(new RemediationScheduler.Task(s.signatureId, profit, deadline));
317         }
318         RemediationScheduler.ScheduleResult result = RemediationScheduler.schedule(tasks);
319         System.out.println("\n[CO1 | Job-Sequencing / NP-hard-scheduling class] "
320                 + result.scheduledInOrder.size() + " of " + tasks.size()
321                 + " candidate remediation tasks fit into the available slots "
322                 + "(total profit=" + result.totalProfit + ")");
323         return result;
324     }
325
326     // ------------------------------------------------------------------
327     // CO4 - network flow (Edmonds-Karp) + max-flow/min-cut duality
328     // ------------------------------------------------------------------
329
330     private static EngineerAssignment.Result runEngineerAssignment(List<ReportGenerator.Summary> summaries) {
331         Map<String, String> categoryOf = new HashMap<>();
332         categoryOf.put(key: key: "NULL_POINTER", value: value: "BACKEND");
333         categoryOf.put(key: key: "TIMEOUT", value: value: "BACKEND");
334         categoryOf.put(key: key: "CONN_REFUSED", value: value: "INFRA");
335         categoryOf.put(key: key: "OOM", value: value: "INFRA");
336         categoryOf.put(key: key: "DISK_FULL", value: value: "INFRA");
337         categoryOf.put(key: key: "AUTH_FAILED", value: value: "SECURITY");
338         categoryOf.put(key: key: "NOT_FOUND_404", value: value: "MISC");
339         categoryOf.put(key: key: "DEPRECATED_API", value: value: "MISC");
340         // Any FUZZY-CLUSTER-* id not listed above falls back to "MISC" via getOrDefault.
```

Fig. 6: Main.java – interval DP merging (CO3), job-sequencing remediation scheduling (CO1) and start of engineer assignment (CO4), lines 283–340



```
        categoryOf.put(key: key: "NOT_FOUND_404", value: value: "MISC");
        categoryOf.put(key: key: "DEPRECATED_API", value: value: "MISC");
        // Any FUZZY-CLUSTER-* id not listed above falls back to "MISC" via getOrDefault.

        List<EngineerAssignment.Engineer> engineers = List.of(
                new EngineerAssignment.Engineer("Asha (backend)", 3, Set.of(e1: e1: "BACKEND")),
                new EngineerAssignment.Engineer("Ravi (infra)", 2, Set.of(e1: e1: "INFRA")),
                new EngineerAssignment.Engineer("Kiran (security)", 2, Set.of(e1: e1: "SECURITY")),
                new EngineerAssignment.Engineer("Priya (on-call/misc)", 4, Set.of("MISC", "BACKEND", "INFRA", "SECURITY"))
        );

        List<String> demandIds = new ArrayList<>();
        List<Integer> demandCounts = new ArrayList<>();
        for (ReportGenerator.Summary s : summaries) {
            demandIds.add(s.signatureId);
            demandCounts.add(s.frequency);
        }

        EngineerAssignment.Result result = EngineerAssignment.assign(demandIds, demandCounts, categoryOf, engineers);

        System.out.println("\n[CO4 | Network Flow / Edmonds-Karp] Ticket assignment: "
                + result.totalAssigned + " of " + result.totalDemand + " error tickets assignable "
                + "within today's engineer capacities.");
        for (Map.Entry<String, Integer> e : result.assignedPerEngineer.entrySet()) {
            System.out.println("     " + e.getKey() + " -> " + e.getValue() + " ticket(s) assigned");
        }
        if (result.bottleneckEngineers.isEmpty()) {
            System.out.println(x: x: "   No engineer capacity is fully saturated (no bottleneck in the min cut).");
        } else {
            System.out.println("   Min-cut bottleneck(s) (max-flow/min-cut duality - capacity fully used): "
                    + result.bottleneckEngineers);
        }
        return result;
    }

    // ------------------------------------------------------------------
    // CO5 - NP-hard problem recognition (Vertex Cover) + approximation
    // ------------------------------------------------------------------

    private static Set<String> runVertexCoverAnalysis(BucketingResult buckets) {
        ErrorCorrelationGraph graph = new ErrorCorrelationGraph();
        for (List<ErrorEvent> bucketEvents : buckets.eventsByBucket.values()) {
            Set<String> distinctTypesInBucket = new LinkedHashSet<>();
            for (ErrorEvent e : bucketEvents) {
                distinctTypesInBucket.add(e.getSignatureId());
            }
            List<String> typeList = new ArrayList<>(distinctTypesInBucket);
            for (int i = 0; i < typeList.size(); i++) {
                for (int j = i + 1; j < typeList.size(); j++) {
                    graph.addCooccurrence(typeList.get(i), typeList.get(j));
                }
            }
        }

        Set<String> cover = VertexCoverApproximation.approximate(graph);
        System.out.println("\n[CO5 | NP-hard recognition + 2-approx Vertex Cover] Correlation graph has "
                + graph.nodes().size() + " error-type node(s) and " + graph.edgeCount() + " correlation edge(s).");
        System.out.println("   Minimal-effort monitoring core set (guaranteed <= 2x optimal size): " + cover);
```

Fig. 7: Main.java – network-flow ticket assignment (CO4) and Vertex Cover analysis (CO5), lines 338–395



```
392         Set<String> cover = VertexCoverApproximation.approximate(graph);
393         System.out.println("\n[CO5 | NP-hard recognition + 2-approx Vertex Cover] Correlation graph has "
394                 + graph.nodes().size() + " error-type node(s) and " + graph.edgeCount() + " correlation edge(s).");
395         System.out.println("   Minimal-effort monitoring core set (guaranteed <= 2x optimal size): " + cover);
396         return cover;
397     }
398
399     // ------------------------------------------------------------------
400     // CO6 - randomised algorithms (Las Vegas + Monte Carlo)
401     // ------------------------------------------------------------------
402
403     private static class RandomizedSummary {
404         final double medianFrequency;
405         final double monteCarloEstimatedRate;
406         final double actualErrorRate;
407
408         RandomizedSummary(double medianFrequency, double monteCarloEstimatedRate, double actualErrorRate) {
409             this.medianFrequency = medianFrequency;
410             this.monteCarloEstimatedRate = monteCarloEstimatedRate;
411             this.actualErrorRate = actualErrorRate;
412         }
413     }
414
415     private static RandomizedSummary runRandomizedAnalysis(List<ReportGenerator.Summary> summaries,
416                                                             List<LogEntry> entries) {
417         int[] frequencies = summaries.stream().mapToInt(s -> s.frequency).toArray();
418         double median = RandomizedAlgorithms.medianOf(frequencies);
419
420         List<String> rawMessages = new ArrayList<>();
421         int actualErrorLines = 0;
422         for (LogEntry e : entries) {
423             rawMessages.add(e.getMessage());
424             if (e.getMessage().toUpperCase(Locale.ROOT).contains(s: s: "ERROR")) {
425                 actualErrorLines++;
426             }
427         }
428         double actualRate = entries.isEmpty() ? 0 : (double) actualErrorLines / entries.size();
429         double sampleFraction = 0.4;
430         double estimatedRate = RandomizedAlgorithms.monteCarloErrorRateEstimate(rawMessages, sampleFraction,
431                 line -> line.toUpperCase(Locale.ROOT).contains("ERROR"));
432
433         System.out.println(x: x: "\n[CO6 | Randomised Algorithms]");
434         System.out.println("   Las Vegas (randomised-select): median error-type frequency = " + median
435                 + " (always exact, expected O(n) time)");
436         System.out.printf("   Monte Carlo (%.0f%% random sample): estimated error-line rate = %.3f "
437                         + "(actual = %.3f, sampling error = %.3f)%n",
438                 sampleFraction * 100, estimatedRate, actualRate, Math.abs(estimatedRate - actualRate));
439
440         return new RandomizedSummary(median, estimatedRate, actualRate);
441     }
442
443     // ------------------------------------------------------------------
444     // CO6 - parallel-algorithm primitives (parallel reduce + prefix sum)
445     // ------------------------------------------------------------------
446
447     private static class ParallelSummary {
448         final long totalSeverityWeightedScore;
449         final long[] cumulativeBucketCounts;
```

Fig. 8: Main.java – Vertex Cover output and randomised algorithms: Las Vegas and Monte Carlo (CO6), lines 392–449

```
451         ParallelSummary(long totalSeverityWeightedScore, long[] cumulativeBucketCounts) {
452             this.totalSeverityWeightedScore = totalSeverityWeightedScore;
453             this.cumulativeBucketCounts = cumulativeBucketCounts;
454         }
455     }
456
457     private static ParallelSummary runParallelAnalysis(BucketingResult buckets, List<ErrorEvent> events) {
458         int[] severityWeights = new int[events.size()];
459         for (int i = 0; i < events.size(); i++) {
460             severityWeights[i] = events.get(i).getSeverity().weight;
461         }
462         long totalScore = ParallelPrimitives.parallelReduceSum(severityWeights);
463         long[] cumulative = ParallelPrimitives.parallelPrefixSum(buckets.bucketCounts);
464
465         System.out.println("\n[CO6 | Parallel Primitives] parallel-reduce total severity-weighted score = "
466                 + totalScore + " (work O(n), span O(log n))");
467         System.out.println("   parallel-prefix-sum of bucketed error counts (cumulative, work O(n log n), span O(log^2 n)): "
468                 + Arrays.toString(cumulative));
469
470         return new ParallelSummary(totalScore, cumulative);
471     }
472
473     // ------------------------------------------------------------------
474     // Extended report section for CO4-CO6
475     // ------------------------------------------------------------------
476
477     private static String renderExtendedReport(EngineerAssignment.Result assignment,
478                                                Set<String> coreMonitoringSet,
479                                                RandomizedSummary randomizedSummary,
480                                                ParallelSummary parallelSummary) {
481         StringBuilder sb = new StringBuilder();
482         sb.append(str: str: "\n--- CO4: Network-Flow Ticket Assignment (Edmonds-Karp) ---\n");
483         sb.append(String.format(format: format: "Total tickets assignable: %d / %d%n", assignment.totalAssigned, assignment.totalDemand));
484         for (Map.Entry<String, Integer> e : assignment.assignedPerEngineer.entrySet()) {
485             sb.append(String.format(format: format: "  %-22s %d ticket(s)%n", e.getKey(), e.getValue()));
486         }
487         sb.append(str: str: "Bottleneck engineer(s) (min cut): ")
488                 .append(assignment.bottleneckEngineers.isEmpty() ? "none" : assignment.bottleneckEngineers)
489                 .append('\n');
490
491         sb.append(str: str: "\n--- CO5: NP-Hard Vertex Cover (2-approximation) ---\n");
492         sb.append(str: str: "Minimal-effort monitoring core set: ").append(coreMonitoringSet).append(c: c: '\n');
493
494         sb.append(str: str: "\n--- CO6: Randomised Algorithms ---\n");
495         sb.append(String.format(format: format: "Median error-type frequency (Las Vegas randomised-select): %.2f%n",
496                 randomizedSummary.medianFrequency));
497         sb.append(String.format(format: format: "Monte Carlo estimated error-line rate: %.3f (actual: %.3f)%n",
498                 randomizedSummary.monteCarloEstimatedRate, randomizedSummary.actualErrorRate));
499
500         sb.append(str: str: "\n--- CO6: Parallel Primitives (Fork/Join) ---\n");
501         sb.append(str: str: "Total severity-weighted score (parallel reduce): ")
502                 .append(parallelSummary.totalSeverityWeightedScore).append(c: c: '\n');
503         sb.append(str: str: "Cumulative bucket counts (parallel prefix sum): ")
504                 .append(Arrays.toString(parallelSummary.cumulativeBucketCounts)).append(c: c: '\n');
505
506         sb.append(str: str: "\n============================================================\n");
507         return sb.toString();
508     }
```

Fig. 9: Main.java – parallel primitives (CO6) and the extended report renderer, lines 451–508

```
508     }
509
510     // ------------------------------------------------------------------
511     // Continuous / real-time monitoring demo
512     // ------------------------------------------------------------------
513
514     private static void runLiveMonitoringDemo(Path originalLog, Map<String, String> patternById,
515                                               Map<String, ErrorSignature> sigById) throws Exception {
516         Path liveFile = Paths.get(first: first: "out/live_demo.log");
517         Files.createDirectories(liveFile.getParent());
518         Files.copy(originalLog, liveFile, java.nio.file.StandardCopyOption.REPLACE_EXISTING);
519
520         System.out.println("\n[Continuous Monitoring] Watching " + liveFile
521                 + " for newly appended log lines (real-time demo, ~3s)...");
522
523         LogMonitor monitor = new LogMonitor(liveFile);
524         Thread watcherThread = new Thread(() -> {
525             try {
526                 monitor.watch(line -> {
527                     LogEntry entry = new LogEntry(lineNumber: lineNumber: 0, line);
528                     Map<String, List<Integer>> hits = RabinKarpMatcher.searchMultiple(entry.getMessage(), patternById);
529                     for (Map.Entry<String, List<Integer>> hit : hits.entrySet()) {
530                         if (!hit.getValue().isEmpty()) {
531                             ErrorSignature sig = sigById.get(hit.getKey());
532                             System.out.println("   [LIVE DETECTED] " + sig.getId()
533                                     + " (" + sig.getSeverity() + ") in line: " + line.trim());
534                         }
535                     }
536                 });
537             } catch (Exception ignored) {
538                 // Watcher thread ends when the JVM exits (daemon thread).
539             }
540         });
541         watcherThread.setDaemon(on: on: true);
542         watcherThread.start();
543
544         java.time.format.DateTimeFormatter fmt = java.time.format.DateTimeFormatter.ofPattern(pattern: pattern: "yyyy-MM-dd HH:mm:ss");
545
546         Thread.sleep(millis: millis: 700);
547         Files.writeString(liveFile,
548                 "\n" + LocalDateTime.now().withNano(nanoOfSecond: nanoOfSecond: 0).format(fmt)
549                         + " ERROR NullPointerException at com.app.LiveDemo.run(LiveDemo.java:9)\n",
550                 StandardOpenOption.APPEND);
551
552         Thread.sleep(millis: millis: 700);
553         Files.writeString(liveFile,
554                 LocalDateTime.now().withNano(nanoOfSecond: nanoOfSecond: 0).format(fmt)
555                         + " ERROR OutOfMemoryError: Java heap space in worker-thread-9\n",
556                 StandardOpenOption.APPEND);
557
558         Thread.sleep(millis: millis: 1200);
559         System.out.println(x: x: "[Continuous Monitoring] Demo finished.");
560     }
561 }
562
```

Fig. 10: Main.java – continuous / real-time monitoring demo, lines 508–562

RESULTS

1. Observations

- The system successfully reads and processes server log files.
- Predefined signatures are detected by the Rabin–Karp scan, and KMP searches the full log corpus for an ad-hoc pattern.
- Repeated occurrences are counted, and errors are categorised by severity (Low to Critical).
- Unmatched lines are grouped by edit distance, and the suffix array + LCP array proposes new signatures.
- Bitmask DP picks signatures within a budget; interval DP merges time buckets into report windows.
- Job sequencing ranks the remediation tasks; network flow assigns tickets and finds the bottleneck engineers.
- Vertex Cover (2-approximation), randomised and parallel algorithms complete the analysis.
- Continuous monitoring detects newly appended log lines in real time.

2. Output Produced by the Program

The summary report contains the **error type**, the **number of occurrences** and the **severity level** of every detected error. While running, the program prints one tagged section per technique, and the same information is written to out/summary_report.txt:

Console tag	What it reports
[CO2 | KMP]	Number and character offsets of occurrences of the ad-hoc pattern in the whole corpus
[CO1 | Edit-Distance]	Number of fuzzy clusters formed from unmatched lines, with an example line per cluster
[CO2 | Suffix Array + LCP]	Candidate new error signatures discovered from unmatched lines
[CO3 | Bitmask DP]	Signatures selected under the monitoring budget, with total cost and total value
[CO3 | Interval DP]	Time buckets merged into report windows and the minimum total cost
[CO1 | Job-Sequencing]	Remediation tasks scheduled and total profit
[CO4 | Network Flow]	Tickets assigned per engineer and the min-cut bottleneck engineers
[CO5 | Vertex Cover]	Correlation graph size and the minimal-effort monitoring core set
[CO6 | Randomised / Parallel]	Median frequency, Monte Carlo error-rate estimate, parallel total score and cumulative bucket counts
[Continuous Monitoring]	[LIVE DETECTED] lines for errors appended to the log while it is being watched

3. Results on data/sample_server.log

The program processed **34 log lines** and detected **30 error events**. The error-type summary produced by the report generator is:

Error type	Count	Severity
OOM	3	CRITICAL
DISK_FULL	3	CRITICAL
NULL_POINTER	5	HIGH
CONN_REFUSED	4	HIGH
AUTH_FAILED	3	HIGH
FUZZY-CLUSTER-0	4	MEDIUM
TIMEOUT	3	MEDIUM
NOT_FOUND_404	3	LOW
DEPRECATED_API	2	LOW

The advanced analyses gave the following results:

Technique	Observed result
KMP full-corpus lookup	"OutOfMemoryError" found 3 times, at character offsets 497, 1034 and 1558
Edit-distance clustering	8 unmatched lines formed 1 fuzzy cluster (FUZZY-CLUSTER-0, 4 lines), e.g. "WARN Cache eviction storm detected on node-7, evicted 5200 keys"
Suffix array + LCP	Candidate new signatures: "WARN Cache eviction storm detected on node-7, evicted 5" (length 55), "INFO Health check OK" and "INFO Server started" (length 20 each)
Bitmask DP	Budget 11 out of 23: NULL_POINTER, CONN_REFUSED, OOM and DISK_FULL selected (total cost 11, total value 65)
Interval DP	2316 time buckets of 5 minutes merged into 7 report windows, minimum total cost 129
Job sequencing	4 of 9 remediation tasks scheduled, total profit 38: slot 1 OOM, slot 2 NULL_POINTER, slot 3 FUZZY-CLUSTER-0, slot 4 NOT_FOUND_404
Network flow (Edmonds–Karp)	11 of 30 error tickets assignable: Asha 3, Ravi 2, Kiran 2, Priya 4; all four engineers are saturated (min-cut bottlenecks)
Vertex Cover (2-approximation)	Correlation graph with 9 error-type nodes and 21 edges; core monitoring set of 8 types (NULL_POINTER, CONN_REFUSED, DEPRECATED_API, TIMEOUT, AUTH_FAILED, OOM, FUZZY-CLUSTER-0, DISK_FULL)
Randomised algorithms	Las Vegas median error-type frequency = 3.0; Monte Carlo (40% sample) error-line rate 0.714 against actual 0.706, sampling error 0.008
Parallel primitives	Parallel-reduce severity-weighted score = 79; parallel prefix sum of bucket counts reaches a cumulative total of 30 events

4. Screenshots

```
PS C:\Users\Lenovo\Documents\DSA PROJECT\logmon> $files = Get-ChildItem -Recurse -Path src -Filter *.java | ForEach-Object { $_.FullName }; New-Item -ItemType Directory -Force -Path out\classes | Out
PS C:\Users\Lenovo\Documents\DSA PROJECT\logmon> java -cp out\classes logmon.Main data\sample_server.log
Loaded 34 log lines from data\sample_server.log

[CO2 | KMP] Full-corpus lookup for "OutOfMemoryError": 3 occurrence(s) at character offsets [497, 1034, 1558]

[CO1 | Edit-Distance] Formed 1 fuzzy cluster(s) from 8 unmatched line(s) (similarity threshold 0.82):
   FUZZY-CLUSTER-0 x4  e.g. "WARN Cache eviction storm detected on node-7, evicted 5200 keys"

[CO2 | Suffix Array + LCP] Candidate NEW error signatures discovered from previously unmatched lines:
   "WARN Cache eviction storm detected on node-7, evicted 5" (len=55)
   "INFO Health check OK" (len=20)
   "INFO Server started " (len=20)

[CO3 | Bitmask DP] Resource-constrained monitoring mode (budget=11 of max 23):
   Selected signatures (total cost=11, total value=65):
     NULL_POINTER[NullPointerException] severity=HIGH freq=5 cost=3
     CONN_REFUSED[Connection refused] severity=HIGH freq=4 cost=4
     OOM[OutOfMemoryError] severity=CRITICAL freq=3 cost=2
     DISK_FULL[Disk full] severity=CRITICAL freq=3 cost=2

[CO3 | Interval DP] 2316 time-buckets (each 5 min) merged into 7 report window(s), min total cost=129

[CO1 | Job-Sequencing / NP-hard-scheduling class] 4 of 9 candidate remediation tasks fit into the available slots (total profit=38)

[CO4 | Network Flow / Edmonds-Karp] Ticket assignment: 11 of 30 error tickets assignable within today's engineer capacities.
     Asha (backend) -> 3 ticket(s) assigned
     Ravi (infra) -> 2 ticket(s) assigned
     Kiran (security) -> 2 ticket(s) assigned
     Priya (on-call/misc) -> 4 ticket(s) assigned
   Min-cut bottleneck(s) (max-flow/min-cut duality - capacity fully used): [Asha (backend) (capacity 3), Ravi (infra) (capacity 2), Kiran (security) (capacity 2), Priya (on-call/misc) (capacity 4)]

[CO5 | NP-hard recognition + 2-approx Vertex Cover] Correlation graph has 9 error-type node(s) and 21 correlation edge(s).
   Minimal-effort monitoring core set (guaranteed <= 2x optimal size): [NULL_POINTER, CONN_REFUSED, DEPRECATED_API, TIMEOUT, AUTH_FAILED, OOM, FUZZY-CLUSTER-0, DISK_FULL]

[CO6 | Randomised Algorithms]
   Las Vegas (randomised-select): median error-type frequency = 3.0 (always exact, expected O(n) time)
   Monte Carlo (40% random sample): estimated error-line rate = 0.714 (actual = 0.706, sampling error = 0.008)

[CO6 | Parallel Primitives] parallel-reduce total severity-weighted score = 79 (work O(n), span O(log n))
```

Fig. 11: Console output of a full run on data/sample_server.log – KMP, edit distance, suffix array + LCP, bitmask DP, interval DP, job sequencing, network flow, Vertex Cover, randomised algorithms and parallel reduce.

```
============================================================
 AUTOMATED LOG ERROR MONITORING SYSTEM - SUMMARY REPORT
============================================================

Total log lines processed : 34
Total error events detected: 30

--- Error Type Summary (Type | Frequency | Severity) ---
ERROR TYPE                      COUNT     SEVERITY
OOM                                 3     CRITICAL
DISK_FULL                           3     CRITICAL
NULL_POINTER                        5         HIGH
CONN_REFUSED                        4         HIGH
AUTH_FAILED                         3         HIGH
FUZZY-CLUSTER-0                     4       MEDIUM
TIMEOUT                             3       MEDIUM
NOT_FOUND_404                       3          LOW
DEPRECATED_API                      2          LOW

--- Optimally Merged Report Windows (Interval DP) ---
FROM     TO        ERR-COUNT
bucket-0   bucket-1           5
bucket-2   bucket-3           8
bucket-4   bucket-5           6
bucket-6   bucket-7           6
bucket-8   bucket-9           4
bucket-10  bucket-2314         0
bucket-2315 bucket-2315         1

--- Prioritised Remediation Schedule (Job-Sequencing) ---
Total scheduled severity-weighted profit: 38
  Slot 1   -> OOM                       (profit=12, deadline<=slot 1)
  Slot 2   -> NULL_POINTER              (profit=15, deadline<=slot 2)
  Slot 3   -> FUZZY-CLUSTER-0           (profit=8, deadline<=slot 3)
  Slot 4   -> NOT_FOUND_404             (profit=3, deadline<=slot 4)

============================================================

--- CO4: Network-Flow Ticket Assignment (Edmonds-Karp) ---
Total tickets assignable: 11 / 30
  Asha (backend)         3 ticket(s)
  Ravi (infra)           2 ticket(s)
  Kiran (security)       2 ticket(s)
  Priya (on-call/misc)   4 ticket(s)
Bottleneck engineer(s) (min cut): [Asha (backend) (capacity 3), Ravi (infra) (capacity 2), Kiran (security) (capacity 2), Priya (on-call/misc) (capacity 4)]

--- CO5: NP-Hard Vertex Cover (2-approximation) ---
Minimal-effort monitoring core set: [NULL_POINTER, CONN_REFUSED, DEPRECATED_API, TIMEOUT, AUTH_FAILED, OOM, FUZZY-CLUSTER-0, DISK_FULL]

--- CO6: Randomised Algorithms ---
Median error-type frequency (Las Vegas randomised-select): 3.00
Monte Carlo estimated error-line rate: 0.714 (actual: 0.706)

--- CO6: Parallel Primitives (Fork/Join) ---
```

Fig. 12: Generated summary report – error-type summary, merged report windows, remediation schedule and the extended CO4–CO6 sections.

```
--- CO6: Parallel Primitives (Fork/Join) ---
Total severity-weighted score (parallel reduce): 79
Cumulative bucket counts (parallel prefix sum): [2, 5, 9, 13, 16, 19, 22, 25, 28, 29, 29, 29, 29, 29, 29, 29, 29,
9, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29, 29
```

Fig. 13: Parallel primitives section of the report (the long cumulative-count array is cropped).

5. Conclusion

The Automated Log Error Monitoring System provides a simple solution for analysing large server log files. It automates the detection of predefined error signatures using Rabin–Karp and KMP string matching, while edit-distance clustering and suffix array + LCP discovery catch similar and previously unknown errors. Error frequency analysis and severity-based categorisation help prioritise critical issues, and dynamic programming, job sequencing and network flow turn the detections into monitoring, remediation and assignment plans. Vertex Cover approximation, randomised and parallel algorithms add provable, sampled and scalable analysis. Continuous monitoring detects new errors in real time, reducing manual effort and

speeding up fault identification. Overall, the system demonstrates how Data Structures and Algorithms can be applied to solve real-world system monitoring problems.

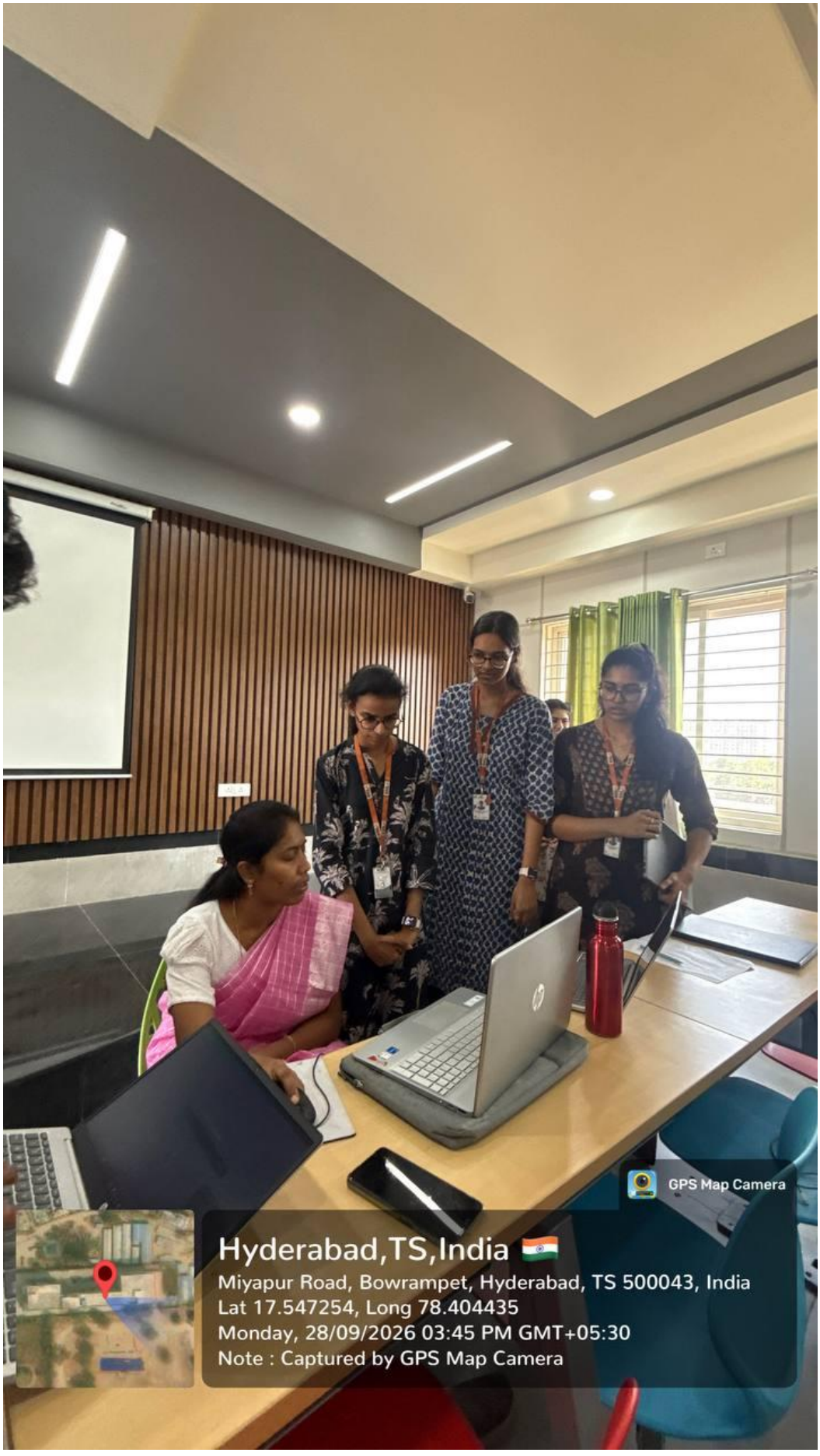
6. Future Scope

- Integration with live server and application log streams beyond file watching.
- Support for a larger number of log formats.
- Automatic promotion of patterns discovered by suffix array + LCP into new known signatures.
- Integration of machine learning for anomaly and error detection.
- Development of a graphical dashboard for visualising error statistics.
- Real-time alert notifications through email or messaging services.
- Support for distributed and cloud-based server monitoring.
- Loading engineer capacities and error categories from configuration instead of fixed values.
- Integration with databases to store historical error information.
- Analysis of error trends to predict possible future system failures.

GEO TAG PICS


GPS Map Camera
Hyderabad,Telangana,India
Miyapur Road, Bowrampet, Hyderabad, Telangana 500043, India
Lat 17.547284, Long 78.403757
Thursday, 20/08/2026 03:46 PM GMT+05:30
Note : Captured by GPS Map Camera


GPS Map Camera
Hyderabad,Telangana,India
Miyapur Road, Bowrampet, Hyderabad, Telangana 500043, India
Lat 17.547214, Long 78.403727
Thursday, 20/08/2026 03:45 PM GMT+05:30
Note : Captured by GPS Map Camera


GPS Map Camera
Hyderabad,TS,India
Miyapur Road, Bowrampet, Hyderabad, TS 500043, India
Lat 17.547254, Long 78.404435
Monday, 28/09/2026 03:45 PM GMT+05:30
Note : Captured by GPS Map Camera


GPS Map Camera
Hyderabad,TS,India
Miyapur Road, Bowrampet, Hyderabad, TS 500043, India
Lat 17.547254, Long 78.404435
Monday, 28/09/2026 03:45 PM GMT+05:30
Note : Captured by GPS Map Camera